

Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

ISO 25010

Software quality								
Suitable	**Efficient**	**Compatible**	**Usable**	**Reliable**	**Secure**	**Maintainable**	**Flexible**	**Safe**
Complete	Time	Co-existence	recognizable	fault-free	confidential	modular	adaptable	operations
Correct	Resources	interoperable	learnable	availability	integrity	reusable	scalable	id risks
Appropriate	Capacity		user error	fault tolerant	non-repudiation	analysable	installable	fail-safe
			engagement	recoverability	accountability	modifiable	replaceable	hazard warn
			inclusivity		authenticity	testable		safe integration
			assistance		resistance			
			self-describe					

Risk

Risk = cost × likelihood

	cheap	expensive
unlikely	Accept	Transfer (insure)
likely	Reduce	Avoid

Security must be built-in from the start

- Complex and subtle
  - You cannot roll-your-own, must use standard solutions
  - If hodge-podge, weakest link determines security
- Security architecture
  - Follow standard
  - Moving target
    - Solns that were secure in 2023, are not in 2025
      - e.g.: RSA public key encryption, no longer secure
- Security design patterns
  - Out-of-scope for ASD

Secure software patterns





3 Main security problems

- Memory safety
  - Read/write forbidden data
  - Rights escalation
  - Code execution
- Code injection
  - SQL injection
  - Command injection
- Authentication
  - Passwords
  - Session management

Key point

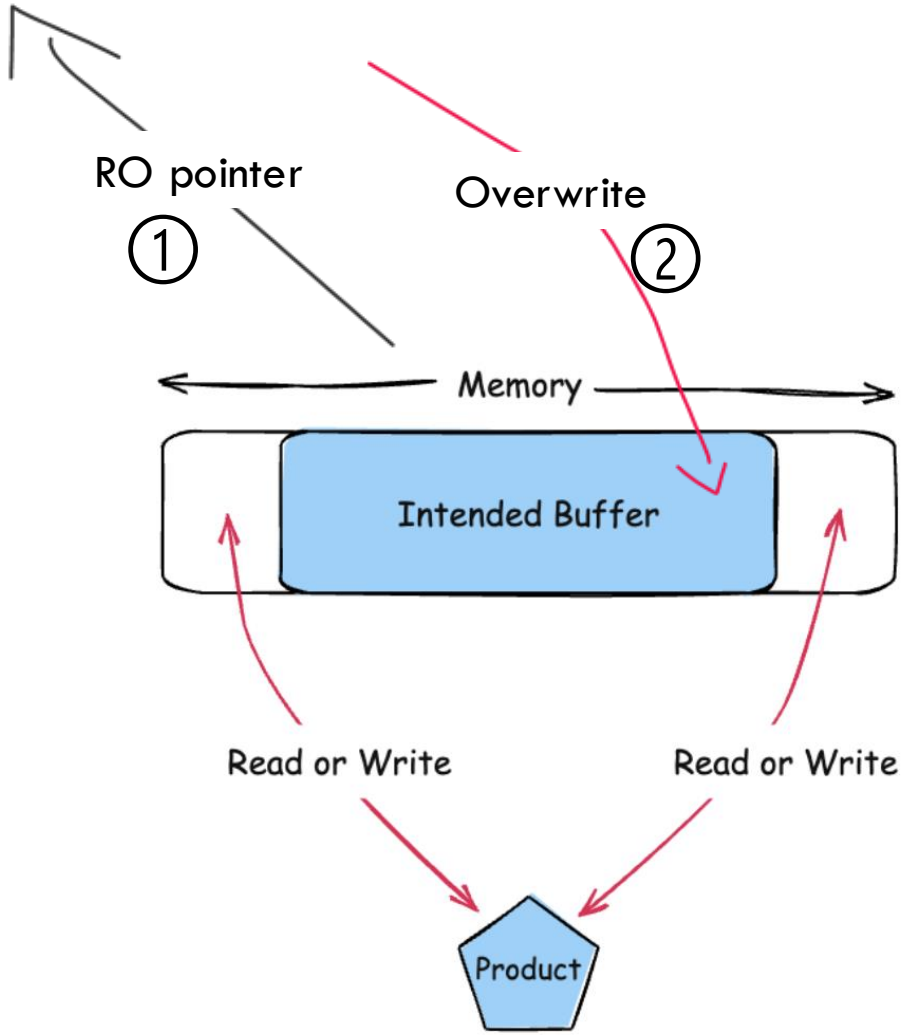
Never roll your own security fix

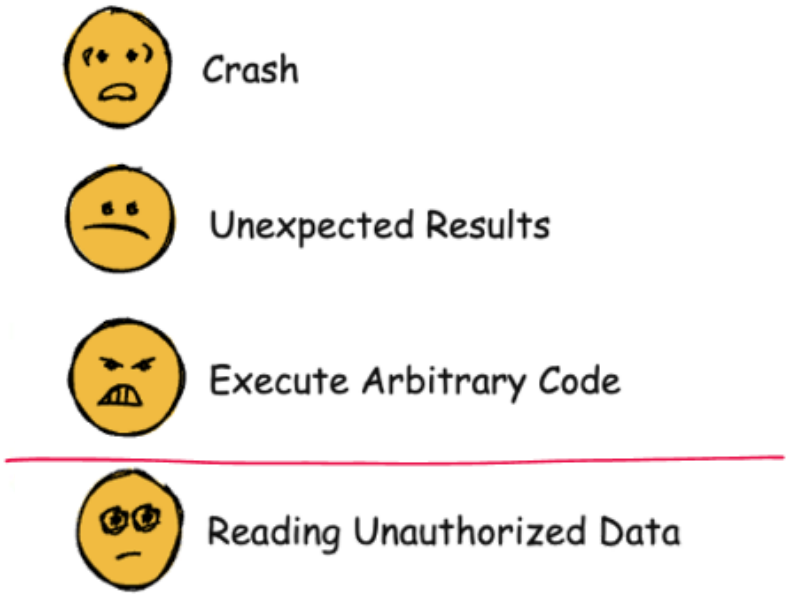
- Always use a known good security pattern
- Every security problem has its own *very specific* fix
- E.g.:
  - SQL injection = code injection
  - XSS = code injection
  - Completely different fixes, zero overlap
- Do not use outdated fixes!
  - E.g.: SQL injection cannot be escaped
  - No matter what your 2001 website says
- 80% secure is not secure

Out of bounds R/W (Buffer overrun), invalid buffer ops

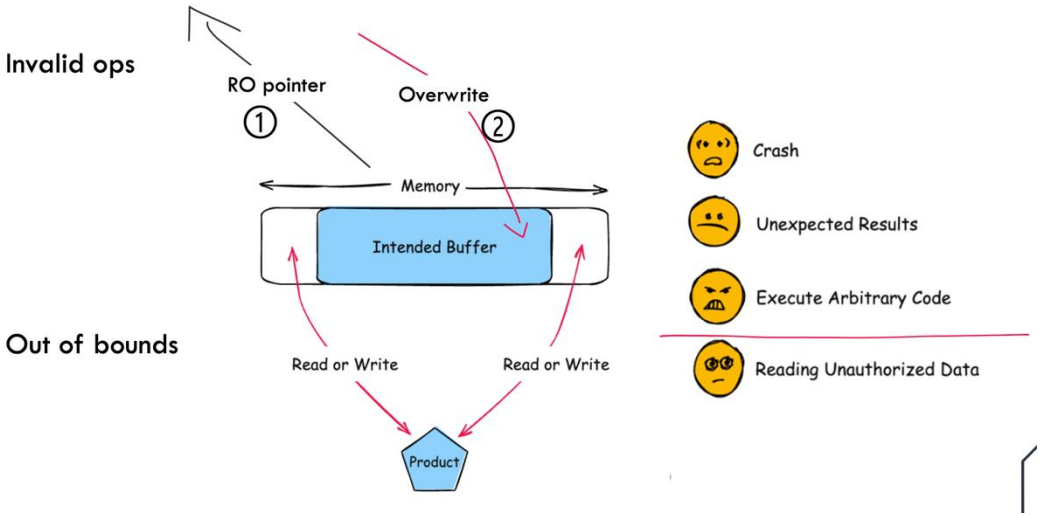
Invalid ops

Out of bounds





Memory safety



Mitigated by:

- Secure languages: Rust/Java
- Insecure languages: use modern techniques
  - C++: use 2023 version "constexpr all the things"
  - C: stop using C, it is and always will be a 1970's lang.
- Compiler hardening flags (stack canaries etc.)
- Code sanitizers (demo)

Stack problems

The stack stores local data

- Buffer overrun changes state
- Also stores function return address
- Overwrite of return address allows arbitrary code execution

Heap problems

Heap contains objects (Object = code + data)

- Object data is nearly always RW, no protection
- Overwrite of virtual method table pointer allows arbitrary code execution
- Many objects live side by side in the same data segment
    - Out of bounds write *unlikely* to trigger exception

C++ demo!!

https://godbolt.org/z/n9jq9dYzb

Code injection

- If code is text
- and input is text
- Then input can be code

Never trust input

- If output is text
- Then output can be code

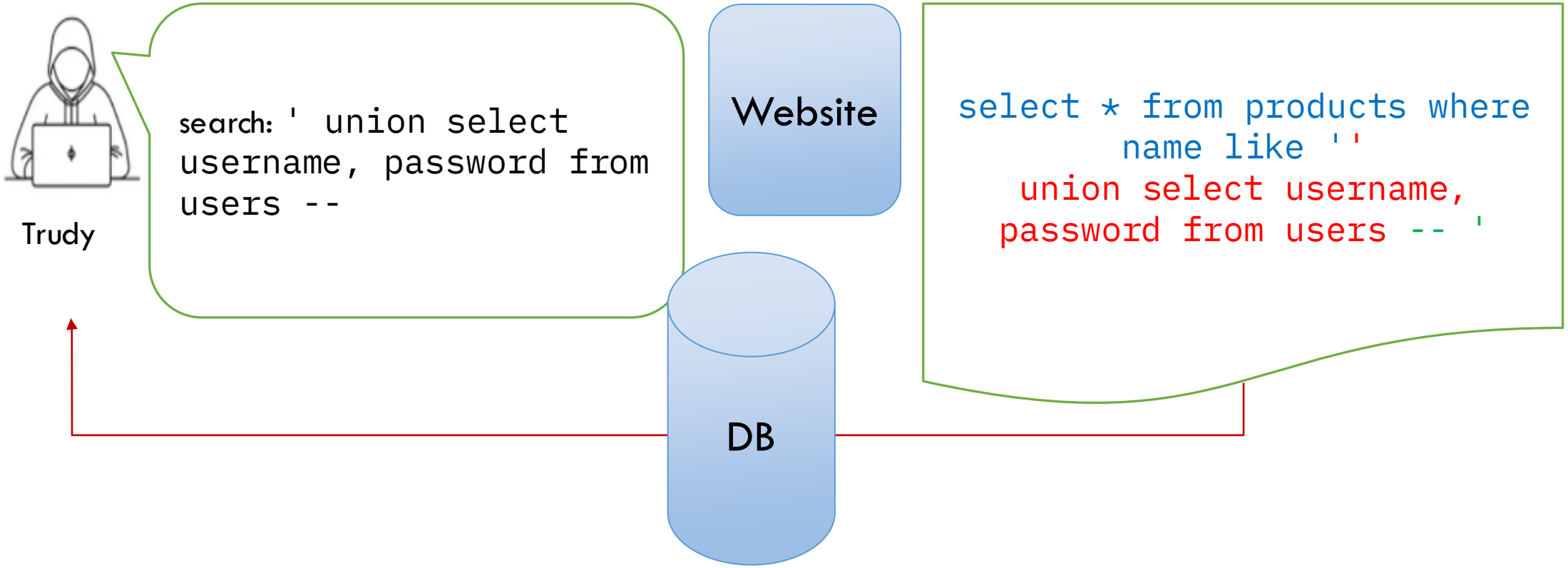
Never trust output!

Security lingo

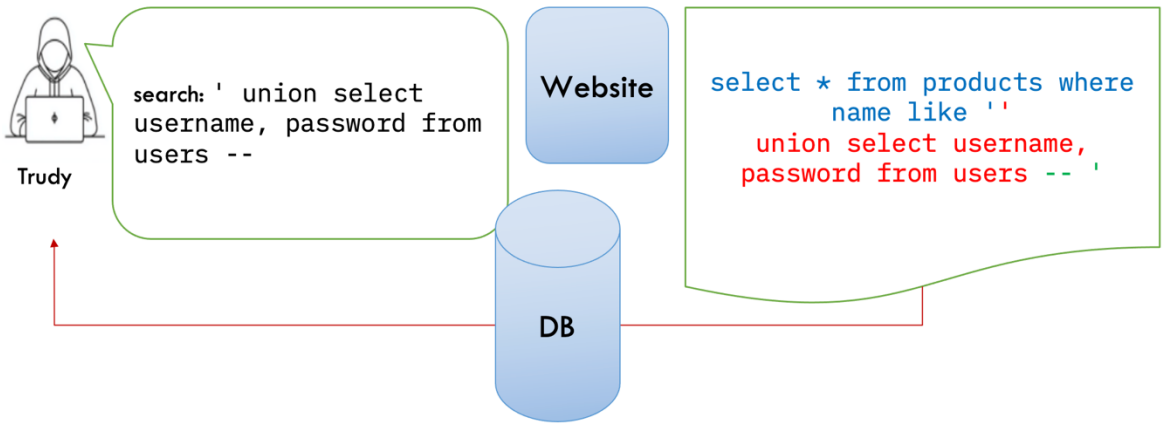
- Alice, Bob, Charlie: normal users, assumed to be noobs
- Eve: eavesdropper if RO, and Man-in-the-middle if R+W
- Mallory: Malicious actor, often a bot
- Trudy: Thief, tries to steal money/credentials

Injection: SQL (do not trust input)

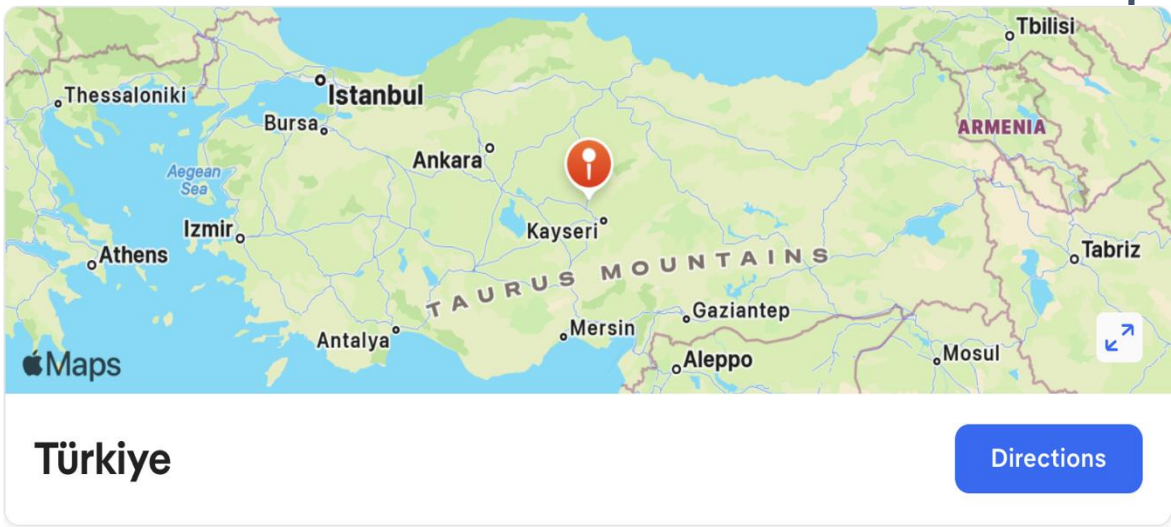
SQL injection



Injection: SQL



There is only one fix!



- You cannot sanitize or escape SQL, it is impossible!
- Use **parameterized queries**
  - never copy-paste SQL
- If you have to use copy-paste, use a whitelist
  - E.g.: dynamic table names

Examples

parameterized queries

✔ safe_query = "SELECT * FROM users WHERE username = ?” 😇

Impossible to fix: W3schools code

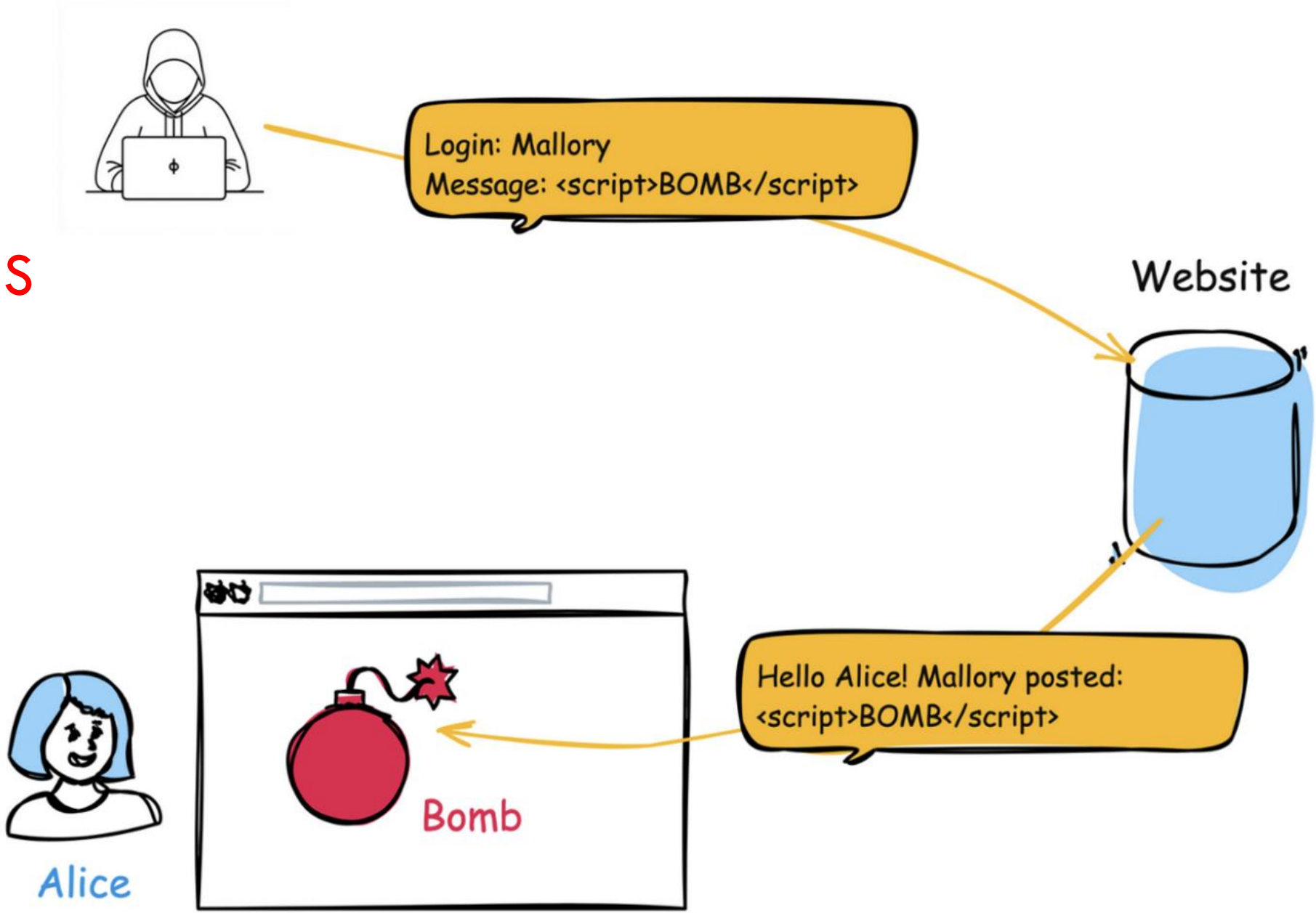
✘ unsafe_query = f"SELECT * FROM users WHERE username = '{user_input}’”

✘ SELECT * FROM Customers WHERE Country = 'Mexico’; 😈

You cannot escape or sanitize SQL

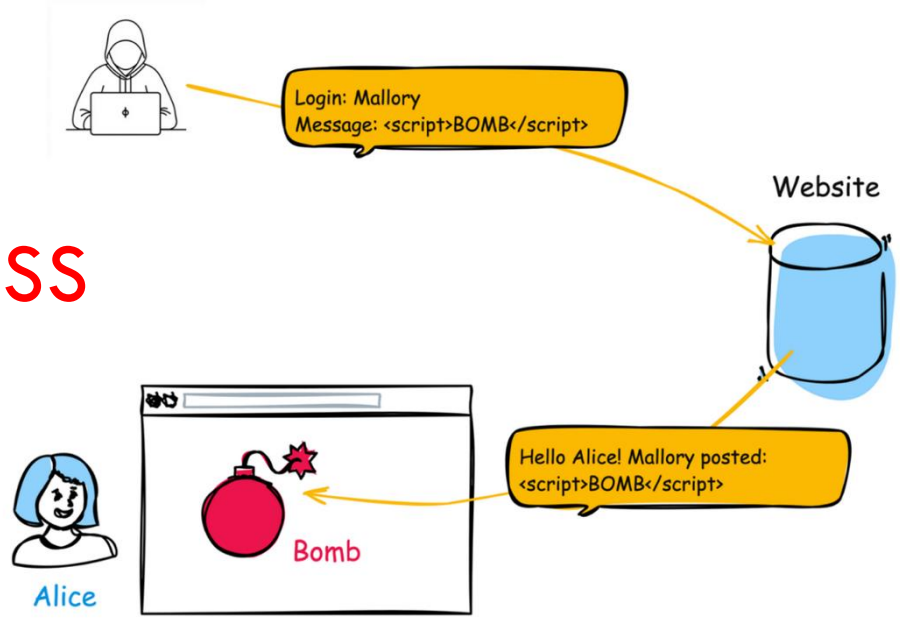
Cross-site scripting XSS (do not trust output)

XSS



Cross-site scripting XSS

XSS



Mitigated by

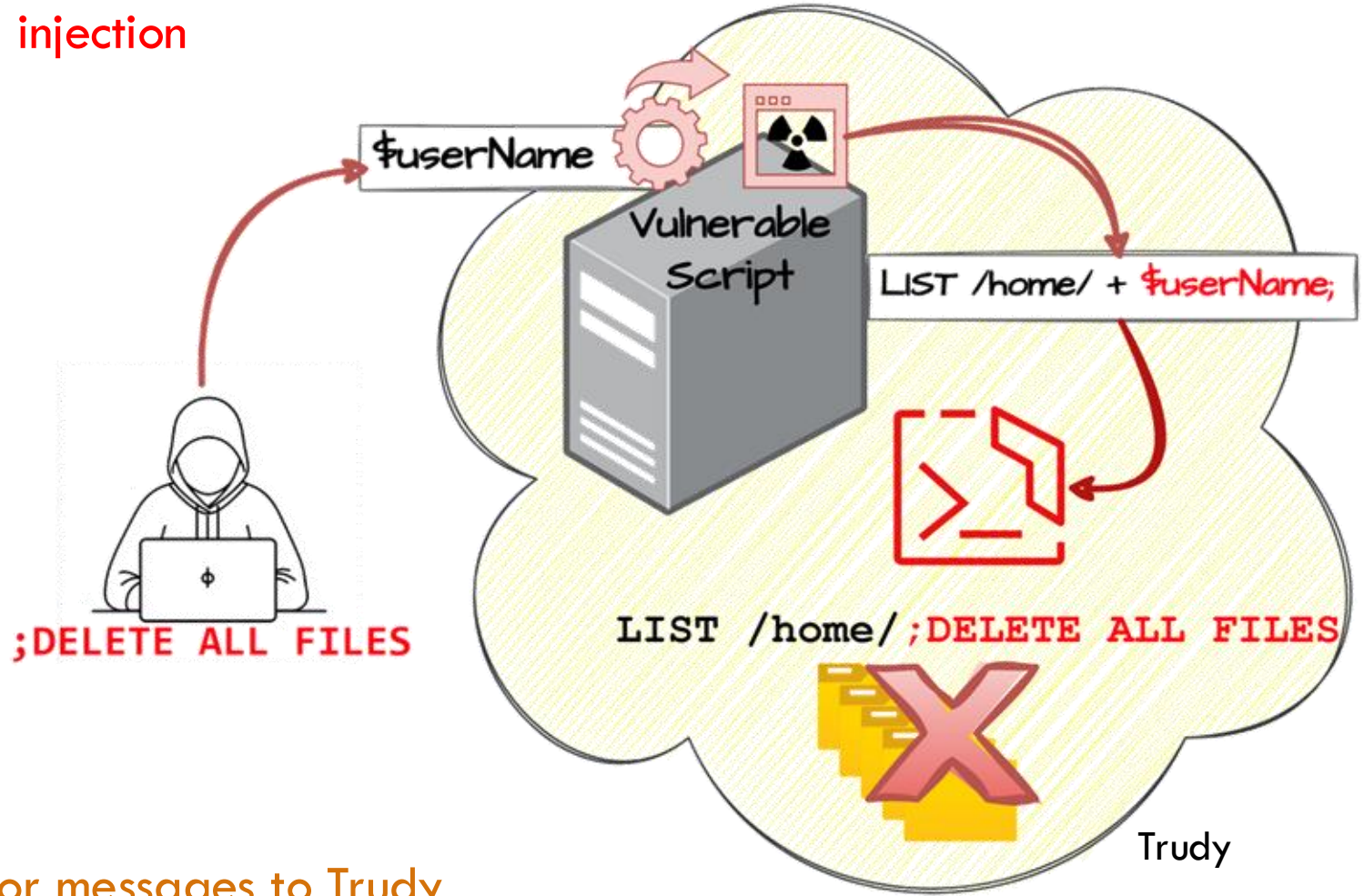
- HTML sanitation
- HTML whitelisting (only allow safe subset)

Whitelist/Blacklist

- PC speak: allow list/deny list
- Blacklists are fundamentally broken
  - Cannot have completeness in an infinite universe
- Whitelists are safe
  - Only allow known safe subset
  - Still non-trivial

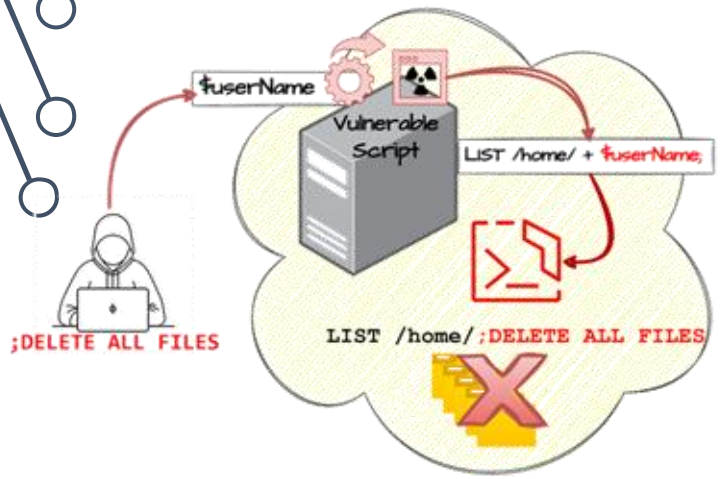
Injection: OS, Code, command

OS/code/command injection



Echoing verbose error messages to Trudy is very helpful for an attacker

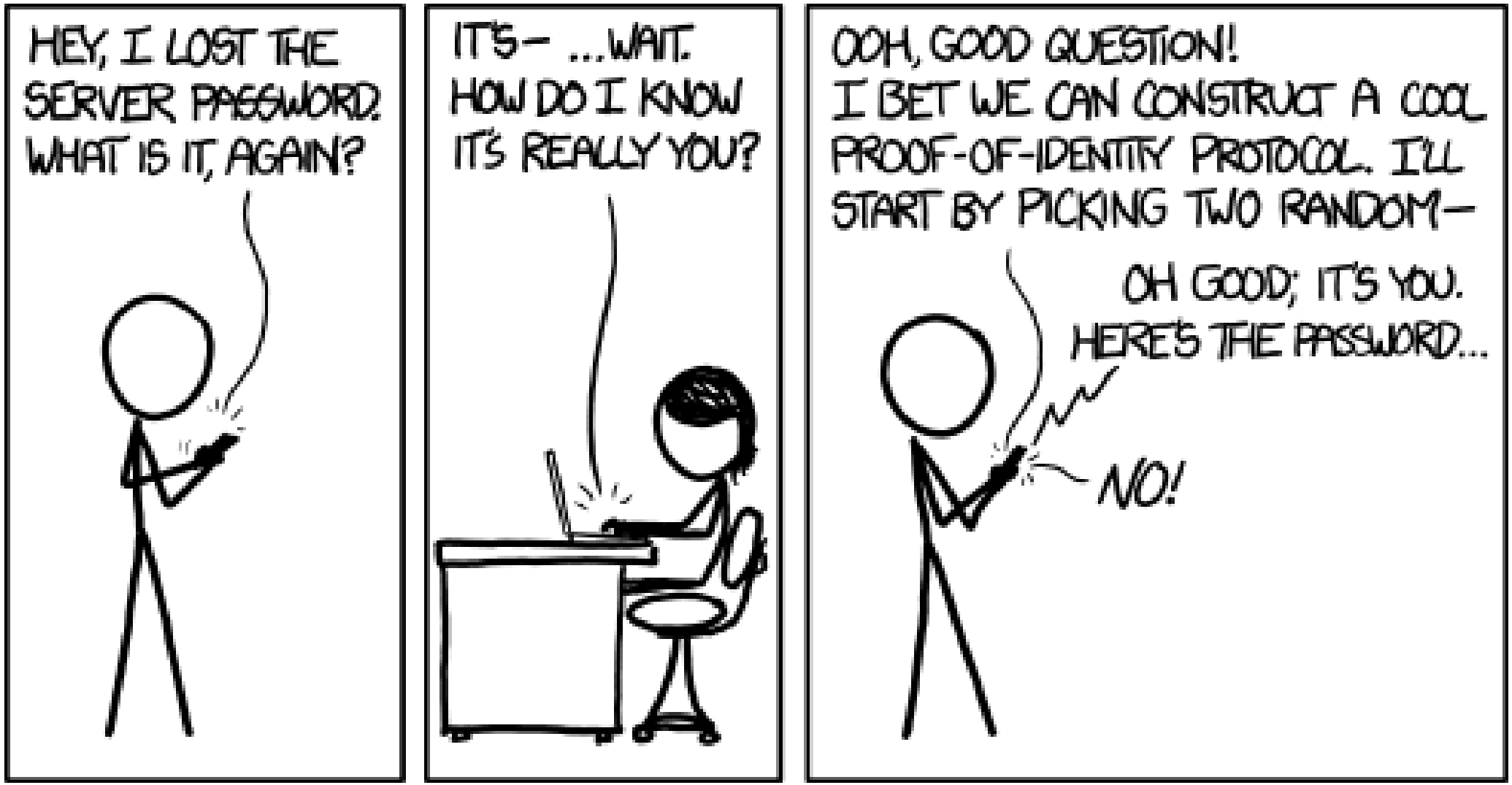
Injection: OS, Code, command



Mitigated by:

- Use parameterized queries
- Escape and sanitize all input
- Never copy-paste input into commands
- Do not echo error messages to users

Authentication



Storing passwords

Database fields:

```
user_id: 007
username: Bont, Johan Bont(es)
salt: strong_random_number(user_id)
passhash: crypto_one_way_hash(password + salt)
```

Why?: https://en.wikipedia.org/wiki/Rainbow_table

Never store passwords in a DB

Slower hashes are better

Storing passwords

Solution: salted one-way hashes (cryptography)

Strengthening:

Slower/stronger hash

Different # of rounds depending on salt

Individual passwords very likely to still be cracked

Website session management

- HTML is stateless
- Non-trivial to track authenticated user

There are (exactly) 3 methods.

Pick one, implement it exactly as prescribed

Python:

```
...
from flask_session import Session


app = Flask(__name__)

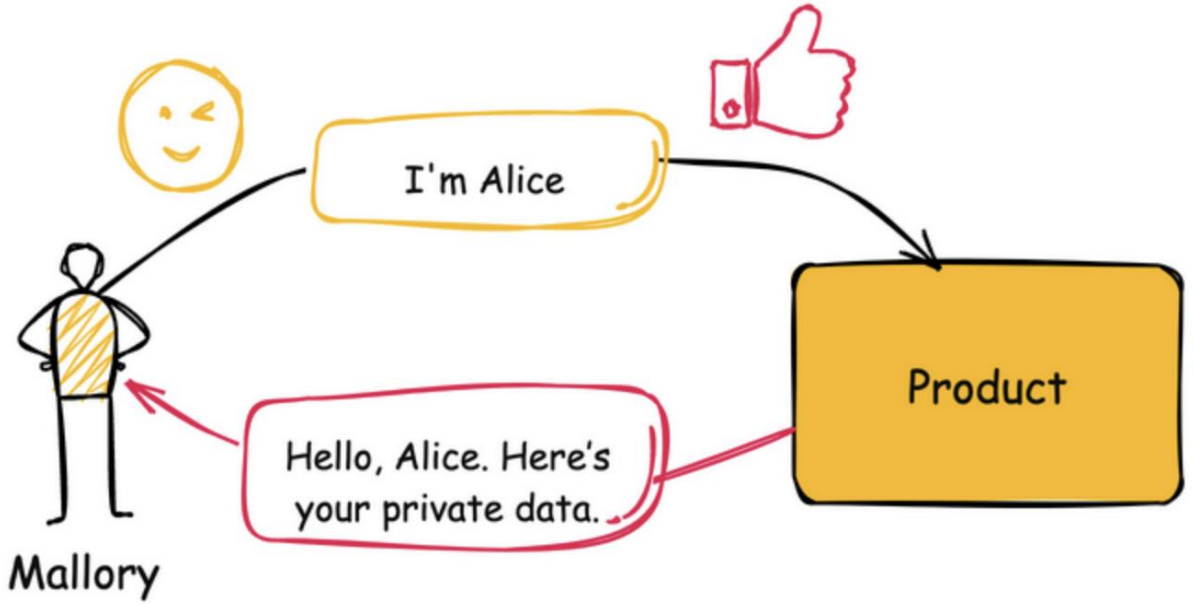

# ---------------- Configuration ----------------
app.config["SESSION_PERMANENT"] = False      # Sessions expire when browser closes
app.config["SESSION_TYPE"] = "filesystem"      # Store session data on the filesystem
Session(app)
```

Java session management

```
import jakarta.servlet.http.*;  //known good soln
...
if (user + password + 2FA = OK) {
  // 2. Erase any existing session
  HttpSession oldSession = request.getSession(false);
  if (oldSession != null) { oldSession.invalidate(); }
  // 3. Generate a clean, secure new session
  HttpSession newSession = request.getSession(true);
  newSession.setAttribute("user", ”role");
  // Store user state safely on the server
  response.sendRedirect(”page.html"); }
else {
  response.sendRedirect("login.html?error=invalid");
} } }
```

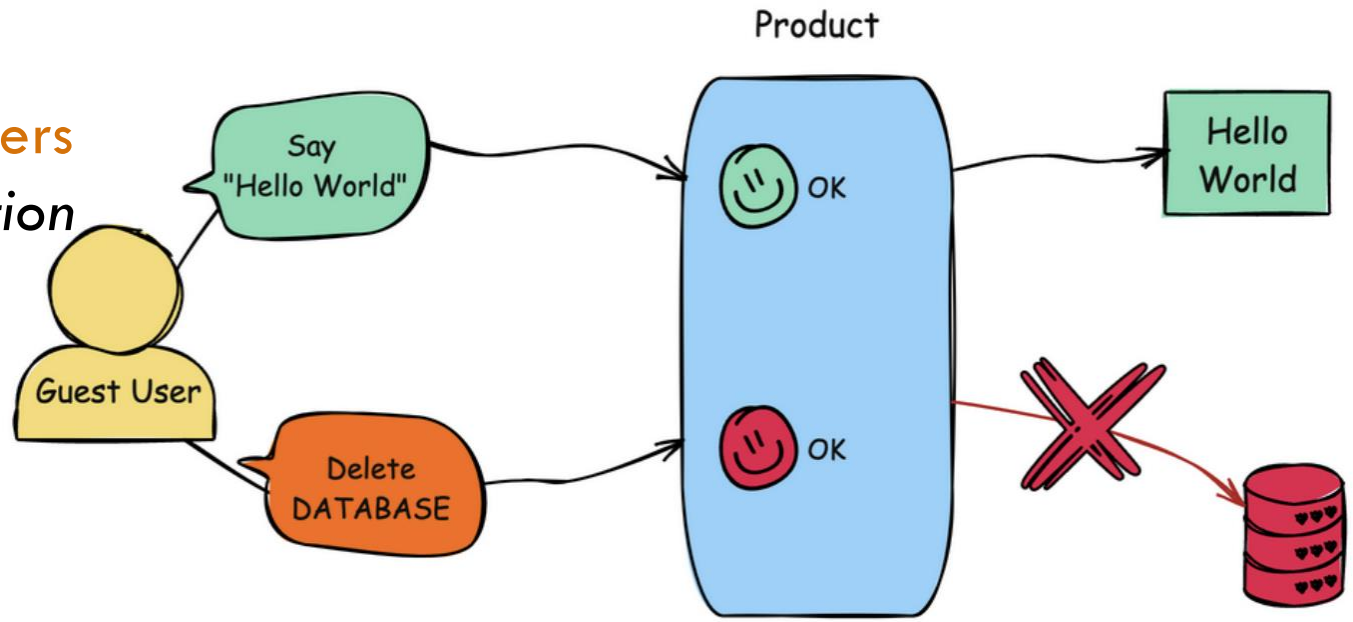
Improper/missing/hardcoded authentication

Weak/missing/hardcoded password or user-id in URL



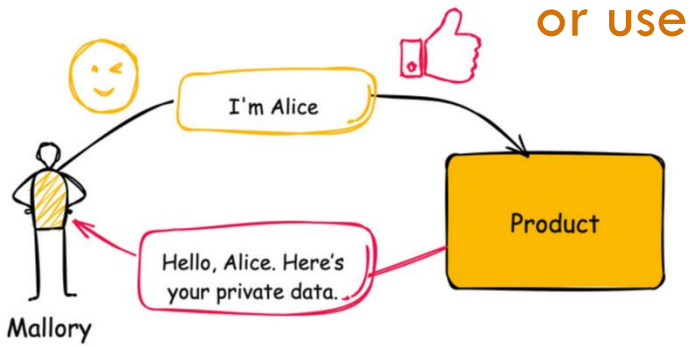
All users are superusers

often combined with injection



Improper/missing/hardcoded authentication

Weak/missing/hardcoded password or user-id in URL



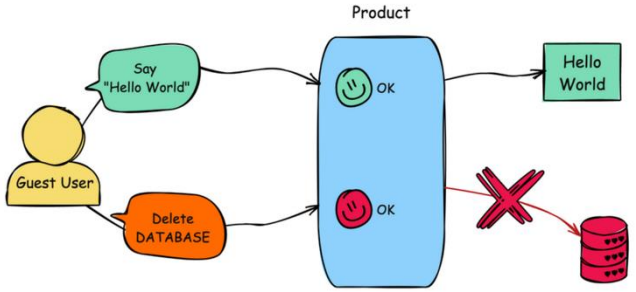
Mitigated by:

- Force change of default password after install
- Never put in backdoors
- Two-factor authentication
- Use standard (= known secure) authentication scheme

Improper/missing/hardcoded authentication

All users are superusers

often combined with injection



Mitigated by:

- Switch on brain (1960 is no more)
- Always disable guest users
- Sandbox untrusted visitors